

Adaptive Detection of Advanced Persistent Threats (APT) with Graph Neural Networks and Rehearsal-Based Continual Learning on Wazuh EDR Telemetry.

Purpose of the Paper:

The main purpose of this paper is to improve the detection of Advanced Persistent Threats in cyber security systems.

APTs are stealthy, long-term attacks that infiltrate systems, escalate privileges, and remain hidden for long periods.

Traditional detection methods struggle because:

- Signature-based systems detect only known attacks.
- Static machine learning models fail when attack

Patterns change

- Concept drift occurs when attack behaviour evolves.
- Catastrophic forgetting happens when models forget previously learned threats during updates.

To address these issues, the paper proposes an adaptive APT detection framework that:

- Uses Graph Neural Network (GNN) to model relationships between system entities.
- Applies Rehearsal-based Continual Learning so the model can learn new threats without forgetting old ones.
- Uses Wazuh EDR telemetry data from endpoints to detect malicious behaviour.

The goal of the project is to create a scalable, adaptive and resilient cybersecurity detection system suitable for Security Operation Centers (SOC) and Endpoint Detection & Response (EDR) environments.

Key Contributions :

1. Graph based threat representation.

- It converts endpoint telemetry data into graphs.
- Captures behavioural relationships between components, which traditional feature-based methods miss.

2. Integration of Graph neural networks.

- The study applies GNN to learn patterns from multi-stage interaction graphs, enabling the model to detect complex multi-stage attacks like APTs.

3. Continual Learning for Evolving Attacks:

- The paper integrates rehearsal-based continual learning, allowing the model to:

- Learn new attack behaviors
- Retain knowledge of previous attacks
- Handle concept drift.

4. Real World Telemetry Evaluation.

- From work is evaluated using:

- Wash EDR telemetry
- Sandbox-executed APT attack scenarios.

* The system achieved F1-Score of 0.98, outperforming traditional static models.

5. Practical SOC/EDC Deployment potential:

The proposed framework is designed to be deployable

in real cyber security env's, especially in:

- SOC monitoring systems
- Endpoint detection platforms
- SIEM tools, (Security Information and Event Management).

Methodology:-

The pipeline consist of 4 stages:

1. Data Collection
2. Graph Construction
3. GNN-based Detection
4. Continual learning with rehearsal

Data Collection:-

Telemetry data is collected from the Wazuh EDR Platform, including:

- Process events
 - File access logs
 - Network Connections
 - User activities
 - System events.
- * Gathers real time, behavioral data from end points
* provide the raw input for attack detection.

Graph Construction:-

Telemetry data is converted into graph structures.

<u>Ex:-</u>	<u>Node type</u>	<u>Example</u>
	Process	chrome.exe
	File	config.txt
	IP Address	192.168.1.5
	User	admin

Edges represents interactions seen as:

- Process → file access
- Process → network connection.

This forms system behavior graph.

- Converts raw logs into a structured behavior graph.
- * Capture relationships between system components.

GNN - Based Detection:

After graph is built, it is processed using a GNN to detect malicious patterns.

GNN Message Passing:

- Each node updates its representation by aggregating information from neighboring nodes.

- The node representation is updated using:

$$h_v^{(k+1)} = \sigma \left(\sum_{u \in N(v)} w_v^u h_u^k \right)$$

h_v = node feature vector

$N(v)$ = neighbors of node

w = learnable weights

σ = activation function

During this process:

1. Nodes collect information from neighbors.
2. Multiple graph layers capture multi-stage relations
3. The network learns patterns of normal vs malicious behavior.

The GNN outputs predictions such as:

- Benign activity
- Suspicious activity
- APT attack.

* Learns Behavioral Patterns of attacks

* Detects multi-stage threats.

Continual Learning with Datasets:

- Cyber attacks constantly evolve, so models must adapt to new threats.
- If we retain the model only with new data, it may suffer from catastrophic forgetting, meaning it forgets previously learned attacks.
- To solve this, the paper uses dataset-based continual learning.

Memory Buffer:

A small set of previous training samples is stored in memory.

This memory contains representative examples of:

- Previous attack patterns
- Normal behaviour.

Training Strategy:-

When new attack data arrives:

- Train the model on new attack data.
- Replay old samples from memory
- Update the model using both datasets.

$$L = L_{\text{new}} + \lambda L_{\text{replay}}$$

L_{new} = loss of new data

L_{replay} = loss from stored samples

λ = balancing parameters

- * Prevents catastrophic forgetting
- * Adapts to evolving attack techniques
- * Improves long-term detection performance.
- * The main goal of this stage is to
 - maintain long-term knowledge of attacks
 - continuously adapt to new threat patterns.

Data Collection



Collect Wazuh EDR telemetry logs

Graph Construction



converts logs → behavior graphs

GNN-Based-Detection



Analyze graphs to detect attack patterns

Continual Learning with rehearsal.

update model while retain past knowledge.

Dataset:-

Benign telemetry : ~12,500 graphs

Malicious APT traces : ~8,300 graphs

Experimental Setup:-

Framework : PyTorch Geometric

GPU : NVIDIA Tesla V100

3-layer GNN

Hidden Dimension : 128

Batch Size - 256

Epochs - 200

Rehearsal buffer - 10%

Models Compared:-

CNN BiLSTM with Attention

Static GNN

Fine-tuned GNN

Replay IDS.

Limitations :-

1. Command-and-Control (C2) detection is weaker.
 - The model performs lower on stealthy outbound communication because traffic may be encrypted or hidden.
2. Scalability Issue:
 - In large enterprises, huge telemetry data increases graph size and memory usage.
3. Adversarial attacks on GNN:
 - Attackers may manipulate graph structure or node features to evade detection.
4. Dependence on data quality:
 - Missing or incomplete logs can reduce graph accuracy.
5. Model trained on one environment may not perform equally well in another.

Results :-

Proposed model achieves F1-Score > 0.98 .

Better Recall than baseline

Minimal latency increase

Better adaptability under concept drift